

name: writing-agent-skill

description: "PHẢI dùng khi: tạo agent/skill mới cho hệ thống KZTEK, cần viết

`C:/Users/nguye/.claude/agents/[name].md` hoặc `C:/Users/nguye/.claude/commands/[name].md` từ đầu, muốn áp dụng quy trình TDD-for-documentation (RED viết test scenario xác nhận vi phạm → GREEN viết định nghĩa → REFACTOR tìm edge case), hoặc user nói 'tạo agent mới', 'viết skill mới', 'cần một agent để...'. KHÔNG dùng khi: chỉ muốn sửa/tối ưu agent đã có sẵn (→ `md-optimizer`), chỉ cần kiểm tra trigger accuracy của agent/skill đã tồn tại (→ `skill-trigger-test`), hoặc chỉ cần sửa nội dung body không liên quan đến quy trình tạo mới."

Skill: writing-agent-skill — Tạo agent/skill mới theo TDD

Học từ obra/superpowers `skills/writing-skills/SKILL.md` — "Writing skills IS Test-Driven Development applied to process documentation."

(xem `docs/research/RESEARCH-superpowers-2026-07-12.md`, §1.3.7)

Nguyên tắc cốt lõi

Viết agent/skill mới theo trực giác dẫn đến 2 vấn đề:

1. Agent/skill không thực sự thay đổi behavior — chỉ là documentation mà LLM bỏ qua.
2. Không biết agent/skill có hoạt động đúng không cho đến khi dùng thực tế.

TDD-for-documentation giải quyết cả hai: **bằng chứng trước, định nghĩa sau.**

Quy trình 5 bước (RED → GREEN → REFACTOR)

Bước 1 — Xác định vấn đề cần giải quyết (tình huống áp lực)

Mô tả rõ:

- **Tình huống gây ra vấn đề** là gì? (VD: agent hay bỏ qua bước scope-check khi user mô tả ngắn)
- **Hành vi vi phạm** cụ thể như thế nào? (VD: Dispatcher routing thẳng WF-FEATURE mà không hỏi priority/scope)
- **Hành vi mong muốn** là gì? (VD: Dispatcher luôn chạy scope-check khi yêu cầu có thể map vào nhiều workflow)

Tình huống : [mô tả kịch bản thực tế gây ra vấn đề]

Vi phạm : [behavior sai cụ thể — KHÔNG chung chung "chất lượng kém"]

Mong muốn : [behavior đúng cần đạt được]

Bước 2 — RED: Viết test scenario và xác nhận vi phạm

Viết 3–5 test scenario mô tả tình huống gây vi phạm:

```
## Test Scenarios (RED — trước khi có agent/skill):
```

```
**Scenario 1:** [User gõ: "..."]
```

```
→ Hành vi hiện tại: [mô tả agent làm gì SAI]
→ Hành vi mong muốn: [mô tả agent nên làm gì ĐÚNG]

**Scenario 2:** [User gõ: "..."]
→ Hành vi hiện tại: [...]
→ Hành vi mong muốn: [...]

...
```

Xác nhận vi phạm (simulation): Với mỗi scenario, kiểm tra agent/Dispatcher hiện tại có xử lý sai không:

- Đọc definition file hiện có (nếu có) → confirm không có rule nào bắt behavior đúng
- Ghi: **Confirmed:** agent hiện tại vi phạm scenario X hoặc **Not confirmed:** agent đã xử lý đúng → không cần skill mới

Nếu không có scenario nào vi phạm → DỪNG, skill không cần thiết.

Bước 3 — GREEN: Viết định nghĩa agent/skill

Dựa trên test scenarios, viết file agent/skill mới:

Với Agent (C:/Users/nguye/.claude/agents/[name].md):

```
---
name: [kebab-case-name]
description: "[Use when/PHẢI dùng khi] [điều kiện trigger cụ thể từ Bước 1]. KHÔNG dùng
khi: [near-miss cases]."
model: [claude-sonnet-4-6 hoặc claude-opus-4-7]
tools: [danh sách tools cần thiết]
---

# [Tên Agent]

## Làm gì
[Nhiệm vụ cụ thể, dựa trên hành vi mong muốn từ Bước 1]

## Red Flags (lý do hay bỏ qua — nhìn nhận lại khi thấy)
| Thought | Reality |
|-----|-----|
| "[Lý do hay bỏ qua]" | "[Phản bác — tại sao lý do đó sai]" |

## Artifact bắt buộc
[File cần tạo ra]
```

Với Skill (C:/Users/nguye/.claude/commands/[name].md):

```
---
description: "[Điều kiện trigger cụ thể — khi nào invoke skill này, không chung chung]"
---

# Skill: [tên]

## Quy trình
[Step-by-step, mỗi bước có output rõ ràng]

## Verification (done gate)
- [ ] [Tiêu chí cụ thể để biết skill đã thực thi đúng]
```

Quy tắc viết description (quan trọng nhất):

- PHẢI bắt đầu bằng "Use when..." hoặc "PHẢI dùng agent này khi..." + điều kiện cụ thể
- Thêm "KHÔNG dùng khi:" + near-miss cases để tránh false positive routing
- Càng cụ thể càng tốt — description mơ hồ = LLM bỏ qua

Bước 4 — GREEN verify: Xác nhận agent/skill mới xử lý đúng

Chạy lại test scenarios từ Bước 2 với định nghĩa mới:

```
## Test Scenarios (GREEN — sau khi có agent/skill):

**Scenario 1:** [User gõ: "..."]
→ Description mới có trigger không?: [Có/Không — trích dẫn phần description liên quan]
→ Behavior đúng chưa?: [Pass/Fail — giải thích]

**Scenario 2:** [...]
→ ...
```

Nếu còn scenario Fail → quay lại Bước 3 sửa definition. KHÔNG đánh dấu GREEN khi còn Fail.

Bước 5 — REFACTOR: Tìm edge case và vá

Đặt câu hỏi: "LLM sẽ tìm có gì để bỏ qua agent/skill này?"

Viết thêm 3–5 "rationalization scenarios" — tình huống mà agent có thể tự thuyết phục là không cần dùng skill:

```
## Rationalization Scenarios (REFACTOR):

**R1:** "Tình huống này có vẻ quá đơn giản để dùng [skill]"
→ Có bị bỏ qua không?: [Có/Không]
→ Fix: [thêm vào Red Flags table / làm rõ description]

**R2:** "User đã nói rõ rồi, không cần [skill]"
→ ...
```

Với mỗi rationalization scenario bị bỏ qua → thêm dòng vào bảng Red Flags trong definition.

Bước 6 — Agent Definition Testing (Reader Testing)

Học từ `skills/doc-coauthoring/SKILL.md` (anthropics/skills) — Stage 3 "Reader Testing": test tài liệu với người đọc chưa có context, KHÔNG phải người đã viết ra nó.

Vấn đề: agent/skill được viết trong session hiện tại — người viết (bạn) có toàn bộ context của cuộc hội thoại dẫn đến quyết định đó. Nhưng khi agent/skill này được gọi thật sự sau này, agent nhận nó sẽ đọc file **hoàn toàn không có context session gốc** ("cold read"). Một definition "rõ ràng" với người vừa viết ra có thể mơ hồ/thiếu sót với agent đọc lần đầu.

Quy trình:

1. Sau khi hoàn thành Bước 4 (GREEN verify) và Bước 5 (REFACTOR), spawn **1 subagent mới, chỉ đưa cho nó file định nghĩa vừa viết** (`C:/Users/nguye/.claude/agents/[name].md` hoặc `C:/Users/nguye/.claude/commands/[name].md`) — KHÔNG kèm theo lịch sử hội thoại, KHÔNG giải thích thêm ngoài nội dung file.

2. Yêu cầu subagent đó tự chạy lại các Test Scenarios (Capability Eval) từ Bước 2, chỉ dựa vào những gì đọc được trong file.
3. Ghi nhận kết quả: subagent có hiểu đúng khi nào trigger không? Có bỏ sót near-miss case nào description chưa liệt kê không? Có bước nào trong quy trình mà subagent hiểu sai ý đồ không?
4. Nếu subagent "cold read" xử lý sai/hiểu nhầm bất kỳ điểm nào → đây là blind spot thật sự (không phải giả định) → quay lại Bước 3 sửa definition, KHÔNG tự cho là "chắc nó sẽ hiểu".
5. Chỉ đánh dấu agent/skill "APPROVED" sau khi Reader Testing pass — khớp yêu cầu Eval-Driven Development (§18.5 CLAUDE.md): $\geq 2/3$ Capability Eval phải pass qua cold-read thật, không phải suy luận của người viết.

Khi nào nên dùng Bundled Resources (scripts/references/assets)

Học từ `skill-creator/SKILL.md` (anthropics/skills) — "Anatomy of a Skill": skill phức tạp nên tách nội dung ra ngoài file chính thay vì nhồi hết vào 1 file, để tránh tốn context khi mọi phần đều bị load dù agent chỉ cần một phần nhỏ.

Đa số skill/agent của KZTEK là single-file và nên **giữ nguyên single-file** — chỉ tách khi có ít nhất 1 trong 3 tín hiệu sau:

1. **Nội dung tham chiếu dài, ít khi cần đọc hết** — VD: bảng tra cứu mã lỗi đầy đủ, danh sách API contract chi tiết mà agent chỉ cần tra 1-2 dòng mỗi lần dùng → tách vào `references/*.md`, file chính chỉ trở đường dẫn.
2. **Có đoạn logic/thao tác lặp lại y hệt mỗi lần chạy, không cần LLM "suy nghĩ" lại** — VD: script chuyển đổi định dạng, script kiểm tra checklist cố định → tách thành `scripts/*.py` hoặc `.sh` để agent GỌI trực tiếp (black-box), không cần nhồi toàn bộ code vào context.
3. **Có asset nhị phân/template cố định dùng lặp lại** — VD: logo, template DOCX, ảnh mẫu → để trong `assets/`, file chính chỉ mô tả cách dùng.

Nếu KHÔNG có tín hiệu nào ở trên → giữ single-file, không tạo cấu trúc thư mục phụ không cần thiết (tránh over-engineering cho skill đơn giản).

Checklist hoàn thành

- [] File agent/skill đã tạo tại đúng vị trí (`C:/Users/nguye/.claude/agents/` hoặc `C:/Users/nguye/.claude/commands/`)
- [] Description bắt đầu bằng "Use when..." / "PHẢI dùng khi..." + điều kiện cụ thể
- [] Có "KHÔNG dùng khi:" với near-miss cases
- [] Có bảng Red Flags (nếu là agent/skill quan trọng hay bị bỏ qua)
- [] Tất cả test scenarios từ Bước 2 đã Pass ở Bước 4
- [] Đã chạy Bước 6 — Agent Definition Testing (cold-read qua subagent mới, không có context session)
- [] Chạy `skill-trigger-test` để verify routing accuracy (xem `C:/Users/nguye/.claude/commands/skill-trigger-test.md`)
- [] Nếu agent mới → cập nhật đầy đủ routing theo §10 CLAUDE.md (org chart, bảng routing, model assignment)
- [] Xuất DOCX+PDF theo §19 CLAUDE.md nếu file .md thuộc docs/

Ví dụ nhanh

Vấn đề: Agent hay tuyên bố "Done" mà không chạy verify lệnh.

RED scenario: "Senior Developer báo 'code xong rồi' mà không paste output build/test."

GREEN: Thêm `## Verification Gate` vào `senior-developer.md` với Iron Law và format output bắt buộc.

REFACTOR rationalization: "Build chạy trong đầu rồi, paste output làm gì mất thời gian" → thêm vào Red Flags.

Keywords

tạo agent mới, viết skill mới, TDD-for-documentation, RED GREEN REFACTOR, test scenario, capability eval, agent definition, skill definition, Reader Testing, cold read, bundled resources, agent introspection, viết định nghĩa agent, viết định nghĩa skill, EDD Eval-Driven Development